İşletim Sistemi Servisleri ve Yapıları (Bölüm 2)

1. İşletim Sistemi Servisleri (Hizmetleri)

İşletim sistemi, uygulamaların çalışması için bir ortam sağlar.

> 💡 ***Mantıksal Benzetme:*** *Bunu bir restoran gibi düşün; mutfak (hardware/donanım) orada durur ama garson (İşletim Sistemi - OS) olmadan sipariş veremezsin. Hangi uygulamaların hangi kaynağı kullanacağını garson yönetir.*

- **Kullanıcı Arayüzleri:**
  - **CLI (Komut Satırı Müşterekliği):** Sadece yazı yazarak komut verdiğin arayüz ("Hacker" ekranı gibi, cmd, bash vb.).
  - **GUI (Grafiksel Arayüz):** Mouse ile pencerelere tıkladığın, hepimizin genellikle kullandığı görsel ekran.
- **Temel İşlemler (OS Servisleri):** Program çalıştırma, I/O işlemleri (yazıcıdan çıktı alma, klavye okuma), dosya/sistem yönetimi (okuma, yazma, silme) ve hata tespiti/yönetimi.

---

2. Sistem Çağrıları (System Calls) ve API - 🚨 KRİTİK KONU!

"Bir uygulama donanıma nasıl erişir?" sorusu İşletim Sistemleri derslerinin en sevilen konularındandır.

Donanıma Neden Doğrudan Erişemeyiz?

> ❓ ***Hoca Sorabilir (Açıklama):*** *"Kullanıcı uygulamaları neden doğrudan donanıma erişemez?"* ✅ ***Cevap:*** *Asıl sebep **Koruma ve Güvenlik**tir. Eğer her uygulama kafasına göre donanıma (hard disk, bellek) erişseydi, bir virüs veya hatalı kod tüm sistemi silebilir ya da diğer programların verilerini çalabilirdi. İşletim sistemi kaynaklara erişimi kontrol ederek koruma (Protection) sağlar.*

System Call (Sistem Çağrısı) Nedir?

Uygulama ile donanım arasında yasal ve güvenli bir sınır kapısı (Filtre) görevi görür. (User Mode'dan -> Kernel Mode'a geçişi tetikler). İşletim sistemi gelen isteği inceler, uygun bulursa erişim hakkı verir.

- **Benzetme:** Bankaya gittiğinde kasaya doğrudan giremezsin. Gişe memuruna (**System Call**) bir talep iletirsen (**API**), o senin adına parayı kasadan (**Hardware**) alır.
- **Sınav Detayı:** Bir dosyayı sadece kopyalamak gibi çok basit görünen bir işlemde bile arka planda **onlarca sistem çağrısı** (dosyayı aç, belleğe oku, hedefe yaz, kapat vb.) arka arkaya çalışır.

API (Application Programming Interface) Nedir?

Programcılar sistem çağrılarını doğrudan, tek tek, manuel yazmazlar. Bunun yerine **API (Örn: Win32, POSIX, Java API)** kullanırlar.

> ❓ ***Hoca Sorabilir (Boşluk Doldurma / Açıklama):*** *"Sistem çağrıları yerine neden API tercih edilir?"* ✅ ***Cevap:*** ***Taşınabilirlik (Portability)*** *ve* ***Kullanım Kolaylığı*** *sağlar. API'ler donanımın karmaşıklığını gizler. Yazdığın kod, o API'yi destekleyen farklı işletim sistemlerinde sorunsuzca çalışabilir.*

---

3. Kavram Karşılaştırmaları (Boşluk Doldurma Bankoları)

Kavram 1	Kavram 2	Temel Farkı / Özelliği

Linker (Bağlayıcı)	**Loader (Yükleyici)**	**Linker**, derlenmiş kodları ve kütüphaneleri birleştirir; **Loader** ise bu birleşmiş çalıştırılabilir programı hafızaya/belleğe (**RAM'e**) taşır (yükler).
Message Passing	**Shared Memory**	**Message Passing (Mesajlaşma)** OS (Kernel) aracılığıyladır. **Shared Memory (Paylaşılan Bellek)** doğrudan bellek (RAM) üzerindendir.
Policy (Politika)	**Mechanism (Mekanizma)**	**Policy**, uygulamanın **"NE yapılacağını"** (what) belirler; **Mechanism**, donanımın/sistemin bunu **"NASIL yapacağını"** (how) belirler.
API	**ABI**	**API** yazılım/kod katmanındaki arayüzdür. **ABI** ise **Donanım seviyesindeki standarttır** (Parametrelerin registerlara nasıl dizileceğini belirler).

---

4. İşletim Sistemi Yapıları (Mimari Farklar)

Hocaların mimari farklarını, "Hangisi avantajlıdır, dezavantajı nedir?" şeklinde sormayı çok sevdiği yerdir.

Mimari Yapı	Temel Mantığı	Avantajı (Neden Kullanılır?)	Dezavantajı
Monolitik (Monolithic)	Her şey (sürücüler, dosya sistemi vb.) tek bir büyük çekirdek parçası içindedir. (Örn: Orijinal UNIX, eski Linux)	Çalışması **çok hızlıdır**. İletişim genel bellek üzerinden anında yapılır.	Bir modül/sürücü bozulursa/çökerse **tüm sistem çökebilir (Kernel panic)**. Hata bulmak zordur.
Mikroçekirdek (Microkernel)	Çekirdek (Kernel) olabildiğince küçültülür (sadece asıl CPU/RAM planlaması kalır). Servisler bağımsız, "kullanıcı modunda" çalışır.	**Çok güvenlidir.** Çoğu servis kernel dışında olduğu için, bir servis çökerse işletim sisteminin kalbi (kernel) ayakta kalır.	Servisler arası iletişim "Mesajlaşma (Message Passing)" ile yapıldığı için çok fazla mod geçişi olur, **performans düşer (Darboğaz/Bottleneck).**
Katmanlı (Layered)	Sistem OSI modelindeki gibi soğan tarzı katman katmandır. Her katman sadece bir altındakini kullanabilir.	Katmanlar izole olduğu için **Debug etmesi (hata bulması ve test etmesi) çok kolaydır.**	Bir işlem birçok katmandan geçmek zorunda olduğu için **performans en düşüktür.**

> 💡 ***Bilgi Kartı: LKM (Loadable Kernel Modules)*** *Modern sistemlerin (bugünkü Windows, Linux vb.) işletim sistemine çalışma anında,* ***sistemi hiç yeniden başlatmadan*** *(dinamik olarak) yeni özellik (genellikle Driver/sürücü) eklemesini sağlayan modüllerdir.*

---

5. Sistemi Başlatma (Booting) ve Hata Yönetimi

Bilgisayarın güç düğmesine bastığında neler oluyor? Bu akış sıralama veya görev sorusu olarak gelebilir:

1. **BIOS / UEFI Yüklemesi:** İlk donanım tespiti yapılır. BIOS sabit diskteki boot block kısmını yükler. Modern sistemlerde BIOS'un yerini çok daha hızlı olan **UEFI** almıştır.
2. **Bootstrap Loader:** Asıl işi yapan budur. ROM veya EEPROM üzerindeki bu küçük program donanımı kontrol edip uyandırır ve işletim sisteminin kalbini **(Kernel'ı) belleğe (RAM) taşır / yükler.**
3. **GRUB:** Özel olarak Linux vb. sistemlerde en popüler ve sık duyacağın gelişmiş cihaz başlatıcısı/bootloader'dır.

Hata Yönetimi (Error Detection) Terimleri

Doğrudan donanım erişimi serbest olsaydı, sızıntı bir program tüm makineyi çökertebilirdi (crash). İşletim sistemi süreci yönetirken sistemin direncini korur. İki önemli kavram:

- **Core Dump:** Bir sürecin (Process/uygulama) hata verip çökmesi anında, bellekte kapladığı alanın (kullanıcı alanı hatası) hata ayıklamak için dosyaya dökülmesidir.
- **Crash Dump:** İşletim sisteminin KENDİSİNDE (**Kernel seviyesi hatası**) meydana gelen ölümcül bir çöküş sonucu durumun kaydedilmesidir (Örn. Mavi Ekran).

📝 Özet: Sınav Öncesi Son Kontrol

- **Sistem Çağrısı:** Uygulamanın User Mode'dan kernel'a geçiş yapıp donanımla konuşma biletidir.
- **Mikroçekirdek:** Çökmelere dayanıklı ve esnektir ama **mesajlaşma (Message passing)** trafiğinden dolayı performans düşüklüğü yaşar.
- **Darboğaz (Bottleneck):** Mikroçekirdekte User mode <-> Kernel mode geçişlerinin yarattığı performans sıkıntısıdır.
- **API Anahtar Kelimeleri:** Taşınabilirlik (Portability) ve programcı için kullanım kolaylığı.
- **Bootstrap / Bootloader:** Sistemi uykudan kaldıran ve Çekirdeği donanımdan RAM'e kopyalayan fedakar program.